

TDD, BDD, and the Stratified Behavioral Refinement Framework in the Era of AI Coding

April 2026

Yixiong Zhou

Founder and Principal Consultant, Marigold Intelligence LLC

Yixiong.zhou@marigoldintelligence.us

Abstract

This paper introduces the **Stratified Behavioral Refinement (SBR)** framework — a new software development lifecycle framework that integrates specification, architecture governance, and testing into a coherent, directed chain designed specifically for autonomous AI code generation. At the centre of SBR is the **trust boundary** — the provability boundary of the test suite, defining the region of system behavior within which autonomous AI generation is trustable. Within this boundary, every behavioral commitment has been formally specified as a failing test before AI generates any implementation; any deviation from the specification is immediately detectable. Outside it, AI generation is unconstrained and unverifiable.

SBR constructs the trust boundary in layers. A three-level SBR test pyramid of unit, component, and contract tests establishes the boundary at every level of implementation, drawing on Gurevich's Abstract State Machine theory as its formal grounding. A BDD Gherkin specification layer, derived from Easy Approach to Requirements Syntax (EARS) requirements, serves as the ground model and provides the grounding proof that the bounded system realizes the specification in integrated execution. Together they form a verification chain in which no layer of AI-generated code exists without a pre-existing, human-specified failing test at that layer.

The paper traces the intellectual lineage of the framework from Beck's TDD through Cohn's test pyramid, XP's acceptance tests, and North's BDD, and shows how SBR synthesises these practices specifically for the AI coding context. It addresses the generate-then-verify anti-pattern, introduces the two-path model separating test derivation from implementation generation, and applies the trust boundary concept to both greenfield development and bug fixing. The SBR framework directly resolves five of the eight unresolved problems identified in the companion paper, and partially addresses three more.

1 Introduction

The companion paper in this series established the methodology context for AI-assisted software development. *Waterfall and Agile in the Era of AI Coding* showed that specification quality is the new velocity constraint — AI multiplies speed when the specification is precise, and multiplies chaos when it is not [1]. It identified a gap that neither methodology choice nor architecture governance fully closes: the fidelity gap between what was specified and what was implemented. A team can have a precise specification, a sound architecture, and a disciplined governance process, and still end up with AI-generated code that looks correct, passes a cursory

review, and fails to realize the actual intent at the level of individual methods, service interactions, or complete user journeys.

This paper closes that gap through the concept of the **trust boundary** — the provability boundary of the test suite, and the governing idea of the SBR framework. The paper builds toward this concept in stages. Parts 1 and 2 establish the true purpose of Test-Driven Development (TDD) and Behavior-Driven Development (BDD), and show why the AI coding era creates both new opportunity and new risk in specification-driven development. Section 4 places SBR in its historical and methodological context, tracing the lineage from XP through BDD and identifying what each prior practice contributed and where each fell short for autonomous AI generation. Section 5 introduces the trust boundary formally and shows how the SBR test pyramid constructs it layer by layer. Section 6 shows how the BDD grounding proof extends verification beyond the trust boundary to the fully assembled system. Section 7 applies the complete framework to a feature development cycle.

The trust boundary is the central contribution of this paper. It also changes how a team reads failure. When AI generation produces a failing test, the failure is not merely a quality gate to clear — it is diagnostic information about the boundary itself. It points either to an implementation that has not satisfied the specification, or to a specification that was not correctly drawn in the first place. These are different problems with different remedies. A team that understands its trust boundary can tell them apart. A team that does not will fix code when it should fix the specification, and fix the specification when it should fix the code.

SBR does not replace TDD or BDD. It gives both practices a precise structural role within a framework whose central purpose is to establish and maintain the trust boundary — the region within which autonomous AI generation is safe, verifiable, and accountable, and within which failure is always a signal worth reading.

This paper is the second of two companion papers addressing AI-assisted software development as a layered discipline. *Waterfall and Agile in the Era of AI Coding* addresses methodology — how development process and sprint structure must adapt when AI accelerates implementation. *TDD, BDD, and the Stratified Behavioral Refinement Framework* addresses verification — how behavior is specified and how the trust boundary that makes AI code generation trustable is built and maintained. The unifying principle across both is that specification precedes generation and tests define the boundary of trust. This paper is written primarily for software engineers and technical leads who will implement the SBR framework in practice.

2 The True Intention of TDD and BDD

2.1 Test-Driven Development: A Design Discipline, Not a Testing Strategy

TDD is often understood primarily as a testing strategy, but its deeper purpose is design — it uses tests as the forcing mechanism [2].

The Red-Green-Refactor cycle is the most familiar expression of this discipline:

1. **Red** — Write a failing test that describes exactly one behavior you intend to implement. The test must fail for the right reason — because the behavior does not yet exist, not because of a configuration error or a poorly written test.
2. **Green** — Write the minimum code necessary to make the test pass. Resist the urge to generalize or optimize prematurely.

3. **Refactor** — Improve the structure of both the code and the test without changing behavior. The passing test suite is your safety net.

The discipline is in the first step. Forcing yourself to write the test first requires you to decide what a function is called, what it accepts, what it returns, and how it fails — before you write a single line of business logic. This is the caller’s perspective, and it consistently produces cleaner, more usable interfaces than writing implementation first.

The original insight, articulated by Kent Beck, is that TDD’s value is not the tests themselves — it is the *thinking* the tests force you to do before you write anything that runs [2]. That thinking requires a cognitive inversion that most practitioners find unnatural.

Rather than asking “*what do I want this code to do,*” TDD asks “*how will I know when it does not do it?*”

The difference is not semantic. The first question produces an implementation. The second produces a falsifiable specification. In human-paced development, a skilled engineer asking the first question can still produce trustworthy software — tacit judgment fills the gaps between the intent and the test. In the AI coding era, that tacit judgment is absent. The generation agent has no intent to draw on — only the specification it is given. With AI generation, this inversion is not optional — it is the prerequisite.

When practiced as intended, a codebase built with TDD discipline has high code coverage as a side effect, not as a goal. The goal is a design that emerged from continuous small clarifications of intent.¹

2.2 Behavior-Driven Development: An Executable Specification Language

BDD, pioneered by Dan North, extends TDD’s discipline to the boundary between the problem domain and the solution domain [3]. Its core insight is that requirements, tests, and documentation should all be the same artifact — readable by engineers, product owners, and non-technical stakeholders alike.

The canonical BDD format, Gherkin, expresses behavior in Given/When/Then:

Table 1. A sample Gherkin feature file

Feature: User Authentication

Scenario: Successful login with valid credentials

Given a registered user with email "user@example.com"

When the user submits the login form with correct credentials

¹ In practice, rigorous specification-first discipline is exercised by a minority of engineering teams. Before AI coding, writing tests first carried a real cost premium — 15–50% overhead — against which immediate implementation pressure consistently won. AI coding shifts this calculus: it reduces the cost of drafting specifications and amplifies the consequences of vague ones. The cultural transition from thinking about implementation to thinking about specification is underway but will not happen quickly. A specific challenge this transition surfaces is the negative case problem: failure modes, error paths, and boundary violations are often implementation-specific and cannot be fully enumerated before any code exists, making it difficult to know when negative case specification is sufficient. EARS unwanted behavior patterns, the RTM, and the Appendix B bug fix cycle together provide bounded answers — but the stopping criterion for negative case specification remains an open research question; see R1 in Appendix E.

Then the user should be redirected to their dashboard
And a valid session token should be issued

Scenario: Failed login with incorrect password

Given a registered user with email "user@example.com"

When the user submits the login form with an incorrect password

Then the user should see an authentication error message

And no session token should be issued

BDD's true purpose is to eliminate ambiguity at the requirements boundary. A scenario written in Given/When/Then cannot be vague without becoming obviously incoherent. The discipline of writing it forces agreement on the nouns, verbs, and outcomes of a feature before any code exists.

BDD scenarios are not documentation that ages. They are living specifications that either pass or fail against the running system. A passing BDD suite is the evidence that the system behaves as specified.

Negative cases — failure modes, error paths, and boundary violations — are a particularly important class of BDD scenario. EARS provides explicit syntax for unwanted behaviors ("If [trigger], the [system] shall [response]"), making known failure modes specifiable before any implementation exists. Failure modes that are implementation-specific and cannot be anticipated upfront enter the trust boundary iteratively through the bug fix cycle (see Appendix B). The question of how many negative case scenarios are sufficient — the stopping criterion for negative case specification — is an open research question; see R1 in Appendix E.

2.3 The Unifying Principle

TDD and BDD share one underlying principle: specification precedes implementation. A specification written before implementation cannot be contaminated by the implementation's assumptions — it represents intent, not behavior that already exists. The gap between the two is where bugs and misdirected implementations live.

In the AI coding era, this principle becomes the primary defense against generate-then-verify drift — the dangerous pattern where AI generates the implementation and tests are written afterward to confirm what the code already does. Generate-then-verify produces tests that validate the implementation's assumptions rather than the specification's intent [4]. SBR is the structural answer to that problem.

3 The AI Acceleration Opportunity

3.1 What AI Does Well in the Specification-Verification Cycle

AI code generation tools are genuinely powerful accelerators — but only when inserted at the right points in the cycle. Their strengths map almost perfectly onto the mechanical, high-friction steps that historically caused developers to skip or defer TDD and BDD work:

The overhead of writing tests first has historically ranged from 15% to 50% of initial implementation cost — a well-documented barrier to consistent TDD adoption [5][6]. But this

framing misses an important nuance: human engineers writing code without pre-written tests are not merely being lazy. They are reviewing and interpreting the specification during implementation, and post-hoc tests capture the design decisions made in that process [6]. AI coding removes this implicit review entirely. The AI has no implementation experience to draw on, no specification intuition to apply, and no design judgment to exercise during generation. Without pre-written tests, the AI has no ground truth at all.

AI can generate test file structures, draft Gherkin scenarios from a feature description, produce step definition stubs, and suggest edge cases that human engineers might miss — capabilities that remove the primary mechanical friction that held TDD and BDD adoption back [7][8]. This is the tailwind that the AI era provides: **the mechanical barrier to specification-first discipline has collapsed.**

The deeper shift is cultural. Before AI coding, the default professional habit was to think about implementation first — what code to write — and treat tests as downstream confirmation of what was already built. That default was rational: the implicit specification thinking that happens during implementation was a genuine form of design work, captured by post-hoc tests [6]. AI coding removes the implicit review and amplifies the output. Engineers are beginning to make the transition from thinking about how to implement to thinking about how to test — from asking “what do I want this code to do” to asking “how will I know when it does not do it.” This transition will not happen quickly — decades of implementation-first culture do not reverse in a single tool cycle. SBR is the structural framework that meets teams at this transition. It provides the ordered workflow that makes specification-first discipline achievable in practice. It also introduces the trust boundary (Part 4) — the concept that makes the value of that discipline legible to practitioners who have not yet fully internalized it.

3.2 The Generate-Then-Verify Anti-Pattern

One of the more consequential failure modes in AI-assisted development is not bad code — it is a false sense of quality. When a developer asks AI to implement a feature and then asks AI to write tests for that implementation, the result is a test suite that achieves high coverage while testing the wrong thing. The tests confirm the implementation’s assumptions rather than verifying the specification’s intent — a pattern that produces what practitioners call the false confidence problem: everything is green, but the bugs are still there, now protected by tests [4][9].

Generate-then-verify is insidious because it looks like good engineering. The coverage numbers are high. The tests pass. The code review shows a responsible-looking suite. What it does not show is that the tests were designed to pass the code that exists rather than to verify the behavior that was intended. It produces implementation first and then writes a specification that matches — the inverse of what a specification is for.

3.3 Why SBR Is the Structural Answer

The Stratified Behavioral Refinement framework prevents generate-then-verify drift by constraining AI code generation inside a trust boundary. The trust boundary is fully defined by the specification and design decisions that precede generation: the failing tests at every pyramid level, the BDD scenarios, and the implementation skeleton. Everything inside the boundary is safe for autonomous generation. Everything outside it requires human judgment.

The specification at each level is the failing test — the Red in the Red-Green cycle. The failing test is the input to AI generation; AI generates the implementation that turns it Green. The test, drafted against the human-approved specification and reviewed before generation begins, is the guarantee that the specification preceded the implementation. AI plays the role of the implementation engine; the human owns the specification that defines what Green means.

This is not an additional process burden on top of development. It is development, restructured so that the specification work that was always required happens in the right order and in an executable form.

This practice has deep theoretical roots. Dr. Yuri Gurevich of Microsoft Research developed Abstract State Machine (ASM) theory to provide the mathematical foundations of software specification — proving that any algorithm can be fully and precisely described as a sequence of state transitions from a ground model down through successive refinements to executable implementation [10][11]. The SBR framework is the practical expression of that theoretical chain: each level of the test pyramid refines the level above it, and the BDD specification layer is the ground model made executable.

The cost structure of this workflow reflects a genuine shift in where human effort concentrates. Since AI drafts test structures, Gherkin scenarios, and step definition stubs under human supervision, the human effort of test authorship moves from production to review. The human's role is to judge whether an AI-drafted test is a genuine boundary marker — a faster and more tractable task than authoring tests from scratch, but one that demands the same specification clarity as the original TDD discipline.

4 Where SBR Sits in the Software Development Lifecycle (SDLC)

4.1 TDD, XP's Acceptance Tests, BDD, and the AI Era

Beck's TDD was always an implementation-level practice. Its discipline of specifying expected behavior before writing the implementation operates at the level of method contracts, class design, and internal structure. Beck never claimed TDD replaced the need for requirements or higher-level design decisions [2][12]. The emergent design philosophy, as he has since clarified, applies to how internal structure develops incrementally through the refactoring cycle — not to the absence of specification upstream.

XP had its own specification layer, separate from unit tests: user stories written collaboratively with an on-site customer, and acceptance tests that the customer and testers wrote together to verify that the system delivered what was promised [13]. This was a deliberate replacement of heavyweight requirements documents with something lighter, more conversational, and more responsive to change [13]. The customer was present to answer questions, clarify stories, and validate behavior continuously. Specification in XP was not absent — it was embedded in conversation and in the ongoing relationship between the team and accessible customers.

Dan North observed a gap that persisted even within this model. Unit tests, operating at the implementation level, were written in developer language and structured around technical concerns. Business stakeholders could not read them, validate them, or use them to verify that the system was delivering the right behavior [3]. North's insight, which led to BDD and Gherkin, was that the specification layer and the test layer needed to converge into a single artifact — one readable by both business and technical participants, and executable against the running system

[3][14]. BDD was not a replacement for TDD or for XP's acceptance tests. It was a more precise, more structured form of the same specification intent, expressed in a language that closed the communication gap between requirements and implementation [14].

In the AI coding era, the conversational specification model that XP relied on faces a structural problem that has nothing to do with XP's values. AI can participate in the specification conversation — drafting user stories, surfacing ambiguities, summarizing discussion, and proposing scenario structures. What AI cannot do is bear responsibility for the outcome. A specification decision made through conversation must pass through human judgment and receive explicit approval before it becomes an executable requirement, a design commitment, or a test that constrains generation [6]. Allowing AI to carry conversational output directly into specification without that approval gate is the same category of risk as generate-then-verify: the system moves fast, looks coherent, and encodes assumptions that no human consciously endorsed.

SBR's response is to bring specification upstream and make it explicit before generation begins — not by returning to heavyweight waterfall documents, but by requiring that BDD Gherkin scenarios be written and committed before architecture begins, and that the full test pyramid be specified and confirmed red before AI generates any implementation. This is the lineage completed: from XP's acceptance tests, through North's Gherkin specification layer, to a workflow in which that specification layer anchors everything that follows.

4.2 SBR as a New SDLC Framework

SBR is a framework that spans the entire software development lifecycle, from requirements through deployment. Its function is to assign each prior practice a precise role in a directed chain, specify the handoff between them, and insert AI generation at the point where it can contribute autonomously without introducing drift.

SBR integrates five practices: EARS requirements, spec-driven methodology, architecture governance, TDD, and BDD. Each addressed a real problem in its time and each remains valid in its own domain. What was missing was a framework that connected them into a single coherent chain, one where each layer fed the next with precision, and where AI coding was woven into the chain at the right level rather than grafted onto it as an afterthought.

SBR provides that connection. The companion paper and this paper are constituent layers within SBR, each governing a different level of the same chain:

Waterfall and Agile in the Era of AI Coding governs the upstream layer — how EARS requirements are produced, how BDD Gherkin scenarios are derived from them, and why upfront behavioral specification is the quality constraint that determines everything downstream [1].

This paper governs the structural and verification layers — how architectural decisions are recorded as ADRs and embodied in the implementation skeleton, how the SBR test pyramid specifies behavior at every level of implementation, how AI code generation is constrained by pre-existing failing tests, and how the BDD Gherkin suite provides the grounding proof that closes the chain.

The governing invariant runs across both layers: **specification and design precede generation at every level**. No architectural decision is left implicit. No implementation begins without a failing test. No AI code generation proceeds without a bounded, human-specified problem space.

4.3 The Twelve-Step Sequence

The twelve steps illustrated below (Figure 1) make this chain concrete. They are not a waterfall phase list — they are the ordered handoffs within a single feature cycle, applicable equally to new features and to extensions on existing code.

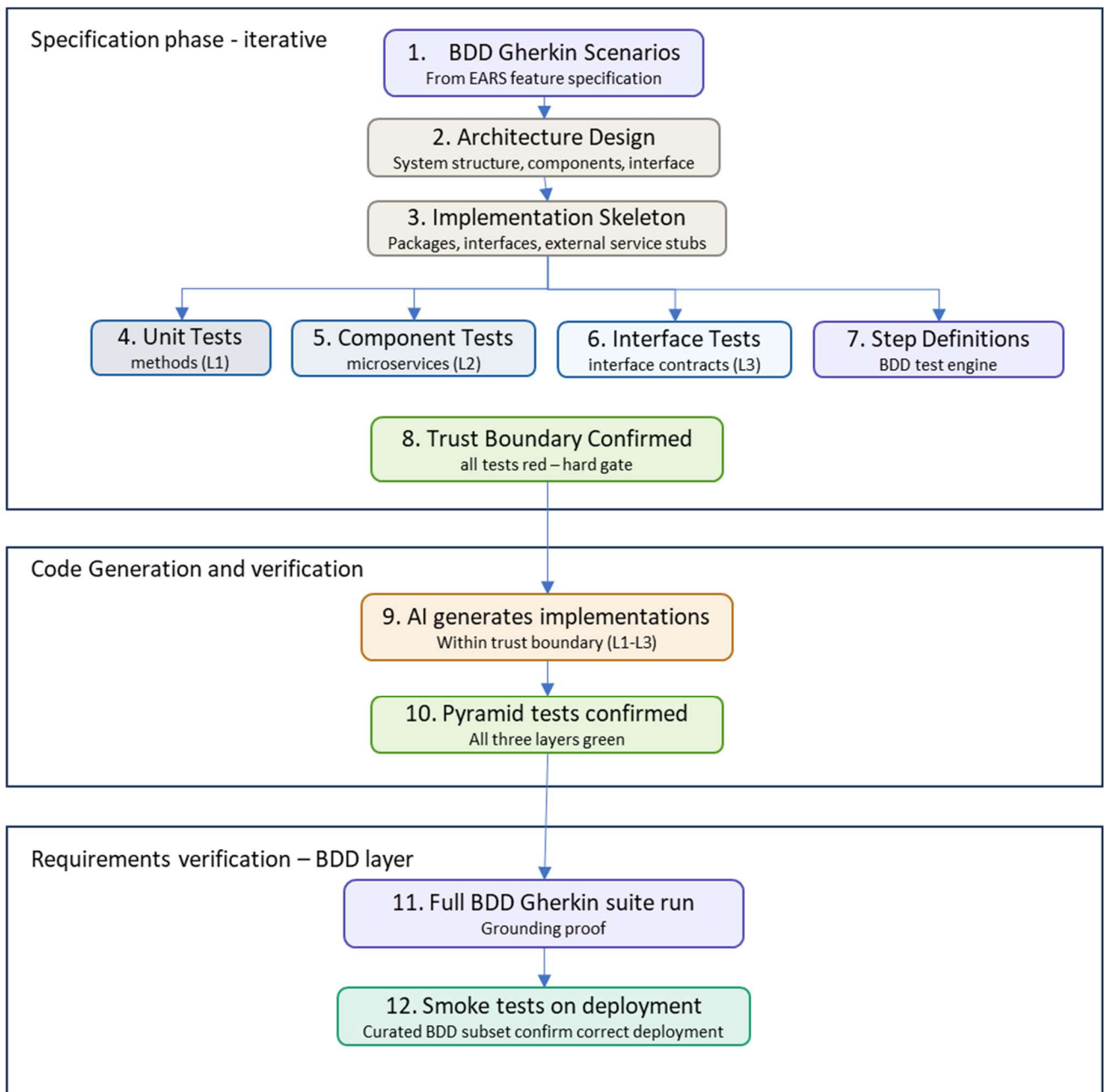


Figure 1. Twelve-step SBR SDLC framework

5 TDD as the Trust Boundary for Autonomous AI Coding

5.1 The Trust Boundary

A test is a formal behavioral statement in a specific sense. Written in the Given/When/Then or Arrange/Act/Assert structure, it encodes a temporal logic safety property: given this precondition, when this action occurs, this postcondition must hold. Because a human writes and approves each test before any implementation exists, the test functions as an axiom — the starting point of a formal system against which implementations are judged, not a description of behavior already present. An implementation that passes that test is a **candidate theorem**: its behavior in that situation is consistent with the axiom. “Candidate” is the precise word — the same axiom set may be satisfied by multiple different implementations, so passing tests bound the space of acceptable implementations without uniquely determining any particular one. The full formal treatment is in Appendix C.

The **trust boundary** is the provability boundary of the test suite. It is the region of the system’s behavior within which correctness has been established as a candidate theorem — behavior that satisfies all test axioms and cannot be contradicted by the current test suite. Within this region, autonomous AI generation is trustable — any deviation from the specified behavior produces an immediately visible failing test.

Outside the trust boundary, the system may behave — but its behavior is unproven. Three distinct conditions produce unproven behavior, and they require different responses:

Undetected incorrectness — a behavior that should have been specified was not. A test that should exist is missing or insufficiently precise. An incorrect implementation exists in this gap, but the system has no means to detect it. This is the classical bug: wrong behavior that a more complete specification would have caught and prevented.

Undefined behavior — a behavior that was never specified at all. No test is missing because none was ever intended. The system’s behavior here is simply unspecified — the formal system has nothing to say about it. Whether the behavior is desirable or not is a question the test suite cannot answer, because no answer was ever written down.

Miss-specified behavior — a behavior that was specified, but specified incorrectly. A test exists but does not correctly express the intended behavior — it passes or fails for the wrong reason. The boundary marker is present but placed in the wrong location. This is not a missing specification; it is a flawed one.

All three conditions produce the same symptom in production: unexpected behavior (Table 2). The trust boundary changes differently in each case — a fact that directly shapes how bug fixes are applied.

A bug fix for undetected incorrectness adds or amends tests to cover the gap — new or corrected statements are added to the formal system and the boundary becomes more complete. A bug fix for undefined behavior introduces new tests for previously unspecified territory — new statements are added to the formal system and the boundary is extended. A bug fix for miss-specified behavior rewrites an incorrectly specified test — the boundary is altered to reflect the correct specification.

Table 2. Trust boundary anomalies

Specification condition	Behavior condition	Trust boundary effect
Incomplete specification	Undetected incorrectness	Boundary becomes more complete
Undefined specification	Undefined behavior	Boundary is extended
Misspecified specification	Misspecified behavior	Boundary is altered

5.2 The SBR Test Pyramid and the Stratified Trust Boundary

Beck’s TDD and Cohn’s test pyramid were introduced independently. TDD (*Test-Driven Development: By Example*, 2002) [2] was a design discipline operating at the unit level — methods, classes, and small modules. The test pyramid (*Succeeding with Agile*, 2009) was an economic and portfolio argument: unit tests are fast and cheap, UI-level tests are brittle and slow, and a healthy test suite should be weighted accordingly [15]. Cohn was describing how to *distribute* tests across levels, not how to *drive design* at each level. The two ideas were complementary but distinct. The industry largely treated them that way — TDD for implementation discipline, the pyramid for portfolio strategy.

The SBR test pyramid builds on Cohn’s distribution argument but serves a different purpose. Fundamentally, it is a stratified trust boundary: a multi-level Abstract State Machine in which each level is independently specifiable, and the tests at that level define the trust boundary for that level of abstraction. SBR extends the test-first discipline to every level of the refinement chain — a direction originated from ATDD [16][17] practices.

Software systems are designed at multiple levels of abstraction simultaneously — methods, service assemblies, and service boundaries are each independent design decisions, each capable of being wrong independently of the others. A trust boundary drawn only at one level leaves all other levels unguarded. With AI generation, every unguarded level is a real risk — AI can generate incorrect code at any level, and each level’s trust boundary must be established before generation begins. SBR formalizes this as a refinement chain grounded in Gurevich’s ASM theory [10][11]: the BDD specification layer is the ground model; each level of the test pyramid is a formal refinement step that draws a distinct layer of the trust boundary. The full treatment is in Appendix C.

The SBR test pyramid composes these three levels into a coherent structure, all code-level, all driven by the test-first discipline, all run continuously during implementation [15]:

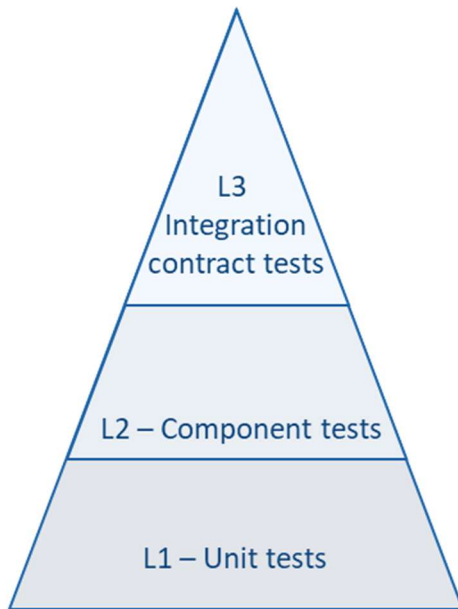


Figure 2. Trust Boundary Tests

Table 3. Three Layers of a Trust Boundary

Level	Test Type	Trust Boundary Drawn	Primary Tools
L1	Unit tests	Method contracts — what each method accepts, returns, and how it fails	JUnit, pytest, Go test, RSpec, etc.
L2	Component tests	Service assembly — how a microservice's internal logic composes and responds	Standard test framework (BDD thinking informs design, not tooling)
L3	Integration / Contract tests	Service boundary contracts — what each service promises to its consumers	Pact, Spring Cloud Contract

Each level draws a boundary no other level can draw. A method that honors its contract (L1 proven) does not guarantee correct service assembly (L2 unproven). Correct service assembly does not guarantee correct interface contracts (L3 unproven). The levels are not redundant — they are independent layers of the trust boundary, each covering territory the others cannot reach.

Before a feature begins, no behavioral statements exist for it — the formal system is silent about its behavior. As specifications are manifested at each pyramid level as tests, new falsifiable statements are added. When all levels pass and the BDD suite confirms grounding, the full set of statements for the feature has been established and verified.

5.3 A Note on Component Tests

Component tests occupy a position in the test pyramid that is frequently unnamed or inconsistently named, which is why they are often confused with either unit tests or integration

tests. In a microservice application, a component test spans the full internal stack of a single service, from the API controller through the service layer to the repository layer, with all external dependencies mocked or stubbed. The database driver is mocked. Downstream service clients are stubbed. But the routing, the controller logic, the service logic, the domain model, and the repository interface are all real, exercised code.

These are also called subcutaneous tests (running just below the API surface), service tests (in the microservices community), or slice tests (in the Spring Boot ecosystem) [18]. The name matters less than the scope: one service, all internal layers, all external I/O mocked.

Component tests are written in the team’s standard test framework, not in Gherkin. But BDD’s Given/When/Then thinking is a valuable design discipline at this level — it forces the developer to define the expected behavior before writing the test code. The mindset is BDD; the tooling is the same unit test framework used at L1. (*The formal basis for Given/When/Then as a specification structure is covered in Appendix C, §C.5.*)

5.4 AI Generation Within the Trust Boundary

With the trust boundary established at all three levels, AI generation operates within a precisely bounded problem space. The boundary is not a description of what AI should do — it defines the extent to which humans can trust what AI has done. Any implementation that deviates from the specified behavior at any level produces a failing test immediately.

At **L1**, the trust boundary covers method contracts. The input to AI is a failing unit test with a precise method signature and assertion. AI generates the method body. It has no latitude to make design decisions — only implementation decisions within the contract the test specifies. If the generated implementation is wrong, the test fails.

At **L2**, the trust boundary covers service assembly. The input to AI is a failing component test that specifies the service’s expected behavior at its API boundary, with mocks defining what external dependencies shall receive and return. AI generates the service wiring — the controller, service class, and repository calls. If the assembly is wrong, the component test fails.

At **L3**, the trust boundary covers service interface contracts. The input to AI is a consumer-driven contract that defines what each service expects from its providers [19][20]. AI generates the adapter code and interface implementations. If a contract is violated, the contract test fails.

Once all three levels are green, the trust boundary covers the full implementation. The BDD specification layer then runs — not as a further layer of the trust boundary, but as the grounding proof that the bounded system realizes the specification it was built from. This is covered in Part 5.

The result is a system where every line of AI-generated code was produced within a human-specified, human-reviewed trust boundary. The generate-then-verify anti-pattern has no natural entry point in this structure: there is no point at which AI generates code without a pre-existing failing test defining the boundary it must satisfy.

Figure 3 shows the complete pipeline schematically, from EARS requirements through verified system, with the trust boundary bracketed around the three pyramid levels.

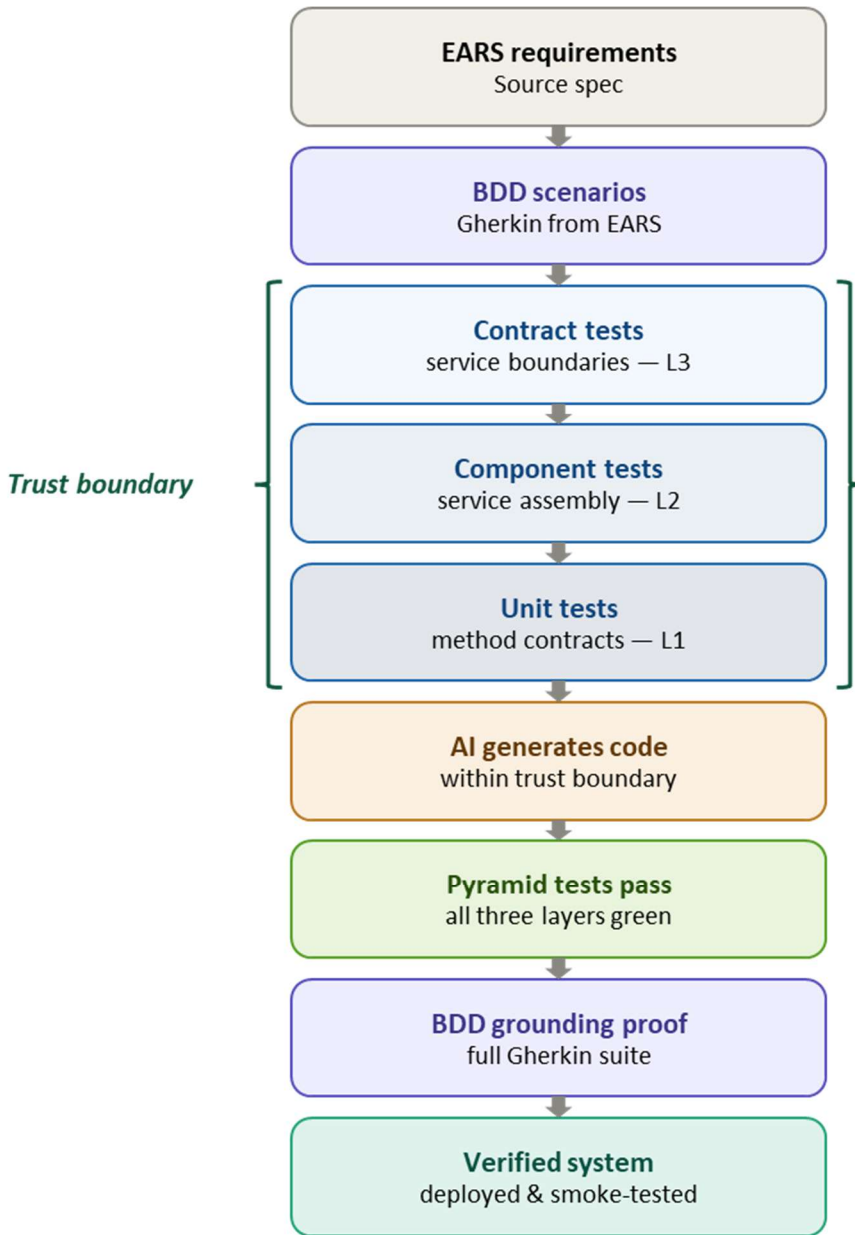


Figure 3: The SBR specification-to-verification pipeline.

5.5 Hypothesis Testing as the Discipline That Keeps the Boundary Honest

The trust boundary is only as strong as the tests that draw it. A test that passes for the wrong reasons — one that does not genuinely detect the absence of the behavior it claims to specify — is a false boundary marker. It gives the appearance of a proven region while leaving the implementation unverified.

This is where TDD’s hypothesis testing discipline becomes essential. Every test is a hypothesis: *the system shall behave this way*. For the hypothesis to be genuine, it must be falsifiable — the test must be capable of failing when the behavior is absent. The test must be red before the

implementation exists. This is TDD's equivalent of Karl Popper's falsification requirement: a hypothesis that cannot be proven false is not a scientific hypothesis [21]. A test that cannot fail is not a boundary marker — it is an illusion of one.

When a test fails after AI generation, there are two hypotheses to investigate:

Hypothesis 1 — The implementation is wrong. The AI-generated code does not satisfy the specified behavior. The boundary is real; the implementation has not reached it. Fix the implementation.

Hypothesis 2 — The test specification is wrong. The test does not correctly express the intended behavior. The boundary marker is misplaced. The right response is not to fix the code but to revisit the specification and redraw the boundary correctly.

This distinction reveals something structurally important about the SBR workflow. Tests and implementations are derived from the same design, but they must travel separate paths and be produced by separate agents:

Path 1 — Design to tests: behavioral specifications derived from the design, written as failing tests, encoding *what* the system shall do at each level. Human-approved before AI automated implementation begins.

Path 2 — Design to implementation: design intent communicated to the generation agent through annotations on the skeleton — method-level comments, class-level documentation, package or service README files, references to ADRs. These encode *how* the system shall be structured. The generation agent reads both the failing tests and these annotations, producing implementations that satisfy the tests within the design constraints.

Keeping these paths separate is not optional. An agent that writes tests and then generates implementation against them has contaminated both paths with the same assumptions — and the falsifiability property of the tests is lost. The tests must be written without knowledge of the implementation; the implementation agent must receive design context through a channel that is independent of the tests.

The practical guidance for constructing design annotations at each level of the skeleton is covered in the workflow section (Section 7).

A scientist who dismisses every failed experiment as a design flaw, without considering the theory, will make slow progress. A developer who treats every failing test purely as a code bug, without considering whether the boundary was correctly drawn, risks building a trust boundary that looks complete but is not.

5.6 Boundary Completeness: The Requirements Traceability Matrix

The hypothesis testing discipline addresses test quality — whether each test is a genuine, falsifiable boundary marker. Quality per test is necessary but not sufficient for trust boundary integrity. A test suite composed entirely of strong, falsifiable tests can still leave significant requirements uncovered. The second property the trust boundary requires is completeness: every specified requirement in current scope maps to at least one BDD scenario, and every BDD scenario maps to pyramid tests at the levels its architecture requires.

Requirement coverage is the correct measure of boundary completeness: every EARS requirement currently in scope maps to a BDD scenario, and every BDD scenario maps to

pyramid tests at the levels the design requires. The **Requirements Traceability Matrix (RTM)** is the instrument for tracking this. In the SBR context it serves two related purposes: it establishes the traceability chain from each EARS requirement through its BDD scenarios to its pyramid tests at every level, and it makes coverage gaps visible as explicit empty cells rather than inferences from test failure. Traceability answers whether every artifact can be anchored to a requirement; coverage answers whether every requirement has been verified. The same table provides both readings.

The popular code coverage measures something different: which lines were executed during the test run. It says nothing about whether those lines correspond to behaviors the requirements specify. A codebase can achieve 90% code coverage with no test that verifies a single specified feature correctly — because tests authored against an existing implementation will exercise that implementation’s actual execution paths regardless of whether those paths correspond to anything a user was promised. High code coverage produced through generate-then-verify is exactly this situation: the metric looks healthy; the trust boundary is uncertain.

In the SBR context, the RTM maps each in-scope requirement down through the specification pipeline (Table 4).

Table 4. Sample Requirements Traceability Matrix						
EARS Req ID	Requirement summary	BDD Scenario	L3 Contract Test(s)	L2 Component Test(s)	L1 Unit Test(s)	Boundary status
REQ-001	When valid credentials submitted, system shall issue session token	Scenario: Successful login	AuthService.login(email, password) → {token, userId}	AuthService: valid credentials accepted	validateCredentials() returns token	Complete — all red [pass]
REQ-002	When invalid password submitted, system shall return AUTH_FAILED	Scenario: Failed login	AuthService.login(email, password) → {error: AUTH_FAILED}	Missing	validateCredentials() returns AUTH_FAILED	Incomplete — L2 gap [gap]

REQ-002 has a BDD scenario, an L3 contract test, and an L1 unit test. Code coverage on the relevant methods may appear adequate. The L2 component test is missing — the service-level trust boundary has not been drawn for this requirement. That gap is invisible to code coverage and visible in the RTM. Both requirements carry an L3 contract because the login feature crosses a frontend-to-backend service boundary, and the RPC interface contract must be specified regardless of whether the internal service assembly is fully covered.

A requirement with no BDD scenario is a requirement entirely outside the trust boundary. A requirement with a BDD scenario but no pyramid tests has a system-level boundary marker

without a verification chain beneath it. Both are gaps; they require different remedies. The RTM makes both visible before AI generation begins.

The RTM is not a heavyweight documentation artifact. In practice it is a lightweight table maintained alongside the .feature files in version control, updated as requirements and tests evolve. Its purpose is a single structural claim: every populated cell at every required level is evidence that the corresponding behavioral commitment has been formally specified; every empty cell is an explicit, visible gap.

The current RTM structure assumes all tests trace to an EARS requirement. Some tests, however, are anchored to a design decision rather than a stated requirement — a unit test that verifies a specific algorithm choice, a component test that enforces a layer boundary, or a contract test whose interface shape was determined by an architectural pattern. These tests cannot trace to an EARS requirement. The proposed resolution is a dual-anchor RTM: every test traces to either an EARS requirement or an ADR. Any test that cannot be anchored to either is flagged as unclassified — an explicit visible gap rather than a silent one. This model is analytically coherent but has not been validated in practice. Further discussion is in Appendix E.

Negative case coverage deserves specific attention in the RTM. EARS unwanted behavior requirements map directly to BDD failure scenarios and pyramid tests; the RTM makes their coverage visible as explicitly as positive cases. Implementation-specific failure modes — those that only become knowable through building and operating the system — cannot appear in the RTM at specification time. They enter the trust boundary iteratively through the Appendix B bug fix cycle. The RTM therefore provides bounded completeness: complete for what was knowable when the specification was written, not for all possible failure modes. R1 in Appendix E identifies the stopping criterion for negative case specification as an open research question.

5.7 *Summary: Trust Boundary Integrity*

Part 4 has addressed two distinct properties that together determine whether a trust boundary can be relied upon.

The first is **quality** — whether each individual test is an effective boundary marker. A test that passes even when the behavior is absent is a false boundary marker. The hypothesis testing discipline governs this property: every test is a falsifiable hypothesis, confirmed red before generation begins. The two-path model — tests derived from design independently of implementation — preserves the independence that makes falsifiability possible. When a test fails after AI generation, it signals either that the implementation has not satisfied the specification, or that the specification was incorrectly drawn. Distinguishing the two requires judgment — from a human reviewer or a reliable AI agent acting under human oversight.

The second is **completeness** — whether the test suite covers all in-scope requirements at every required pyramid level. A set of high-quality tests can still leave requirements entirely unverified. The RTM governs this property: it maps each EARS requirement to its BDD scenarios and pyramid tests, making gaps visible as explicit empty cells rather than silent absences. Tracking test failure rates by level and by sprint complements the RTM — a team whose L2 failures persist after L1 failures have resolved has a component specification gap, not a unit implementation problem.

Neither property implies the other, and both are required for a trust boundary that can be relied upon.

6 BDD as Grounding Proof

6.1 *The Grounding Problem*

The SBR test pyramid ensures that each level of implementation is correct in isolation — methods honor their contracts, services are correctly assembled, and service interfaces honor their interaction agreements. But a pyramid of green tests does not guarantee the most important property of a working system: that the complete, assembled system, running in a production-like environment, actually realizes the feature specification that was written in human-readable language at the specification phase.

The gap between “all pyramid tests pass” and “the system works for users” is where many software projects fail. Components that are individually correct can fail to compose correctly. Services that honor their contracts in isolation can produce wrong behavior when assembled with real data and real timing.

A concrete example illustrates how this gap arises. Consider two services, each correctly specified and verified in isolation: Service A is specified to produce a timestamp in UTC format; Service B is specified to accept and process a timestamp. Service A’s component tests confirm it produces a correctly formatted UTC timestamp. Service B’s component tests, using a stubbed Service A, confirm it processes the timestamp correctly. The contract test confirms that Service A’s timestamp format matches what Service B expects. All pyramid levels are green.

At runtime, however, Service B is deployed in a timezone-aware environment that silently interprets the UTC timestamp as local time. Every individual component specification was correct. Every pyramid test passed. Yet, the composition of two correct components produces wrong behavior under real conditions — and no test at L1, L2, or L3 could have detected it, because each operates in controlled isolation.

This is the failure mode the BDD grounding proof is designed to catch. A Gherkin scenario exercising the full user journey through both services in a staging environment surfaces the failure immediately — not because it is more detailed than the component tests, but because it runs the assembled system against real infrastructure and real conditions rather than each component in isolation.

6.2 *What Grounding Means*

A system is grounded when its behavior, verified against real infrastructure in the actual execution environment, realizes the specification. Until that verification runs, the system remains an unverified assertion. A system whose BDD Gherkin suite passes is a grounded system: its behavior has been demonstrated to correctly realize the specified intent in execution. Every failing scenario is a falsified grounding claim — a specific, executable statement about what the system promised to do and did not do. Unlike a failing unit test, which points to a specific method, a failing Gherkin scenario points to a gap in the specification at the system level — either a behavior that was incorrectly specified, or one that was never specified for the assembled system as a whole.

6.3 *The Full BDD Suite vs. Smoke Tests*

The BDD specification layer produces two distinct artifacts. The **full Gherkin suite** covers every EARS requirement as one or more scenarios; it runs against a fully assembled system in a

staging or production-like environment and serves as the authoritative verification that the specification is realized. Because it is comprehensive, it is suited to the end of a sprint cycle and to significant staging deployments rather than to every commit.

Smoke tests are a curated subset of scenarios, typically tagged @smoke, that cover the most critical user journeys and answer a narrower question: is the system alive and correctly wired after this deployment? They run on every deployment and are fast by design [22]. The two artifacts serve distinct purposes: smoke tests provide a deployment health signal; the full suite provides specification coverage.

6.4 The EARS to Execution Pipeline

The complete specification pipeline in SBR runs from natural language requirements through executable verification. EARS — the Easy Approach to Requirements Syntax — is the structured natural language notation that anchors the pipeline at the requirements level [23][24]:

EARS requirement

- Gherkin BDD scenario (BDD specification layer)
- Architecture design and skeleton
- Unit test (L1)
- Component test (L2)
- Interface / Contract tests (L3)
- Step definition and test engine
- AI-generated implementation
- Passing pyramid tests
- Passing BDD suite (grounding proof)

Each step makes the specification more concrete without losing the intent of the step above it. The EARS requirement captures stakeholder intent in structured natural language. The Gherkin scenario translates that intent into an executable behavioral claim. The architecture design and skeleton embody the structural decisions that bound the implementation space. The unit, component, and contract tests make the claimed behavior verifiable at every level of the implementation. The step definitions and test engine then wire the Gherkin scenarios to the assembled system, making the grounding proof runnable. AI-generated implementation fills the skeleton against the pre-existing, failing tests — and the pipeline closes when both the pyramid and the BDD suite pass.

When both the SBR test pyramid and the BDD specification layer pass, the pipeline has successfully carried the stakeholder’s intent from natural language all the way to verified execution. That is grounding.

6.5 When the BDD Suite Fails After the Pyramid Is Green

A BDD suite failure after all pyramid levels pass is one of the most diagnostically valuable events in the SBR workflow. It means:

- Individual methods satisfy their specifications within isolation (L1 green)
- Service assemblies satisfy their specifications within isolation (L2 green)
- Interface contracts satisfy their specifications within isolation (L3 green)
- But the assembled system, running under real conditions, reveals a specification gap that isolated testing could not expose

This pattern points to a gap in the specification itself — an assumption that was implicit in the Gherkin scenario, a precondition that was missing from the Given clause, a timing or ordering dependency between services that none of the pyramid levels could see. The BDD failure is not a code bug — it is specification feedback. The right response is to trace the failure back through the pipeline to find where the intent was lost.

7 The Complete AI-Accelerated SBR Workflow

7.1 Overview

The SBR workflow is not a replacement for the Agile sprint or the Waterfall specification phase — it is the quality layer that operates within whatever cadence the team uses. The workflow below describes a single feature cycle, which applies equally to new feature development and feature extensions on existing code. The boundary between the two is intentionally blurred: extending an existing feature is simply new feature development within the constraints of an existing codebase, and SBR treats both the same way.

The workflow has three phases. The **specification phase** comprises three streams: BDD scenario writing, architecture design and skeleton building, and SBR test pyramid test writing. These streams are iterative and overlapping. Architecture design may uncover missing BDD scenarios; therefore loop back into BDD scenario writing. SBR test pyramid test writing may reveal both. Until the trust boundary is complete and confirmed red, the streams continue to feed each other. At that point the **generation and verification phase** begins: AI generates implementations within the trust boundary the SBR test pyramid established. The **specification verification phase** then establishes that the system is grounded to its specification.

The workflow in this section progressively builds the trust boundary from the specification layer down through the SBR test pyramid, handing AI a precisely bounded problem space in which to operate. The developer thinks at the level of specification and design; AI works within the trust boundary that those decisions establish. The boundary is the interface between the two roles: everything outside it requires human judgment; everything inside it is safe for autonomous generation.

A practical starting point for SBR is to design and specify what the team currently understands well enough to implement. The trust boundary is then built for that scope — no more and no less. As development progresses through Agile sprint cycles, customer demos surface new understanding and requirements mature. Each iteration extends the design scope, and the trust boundary grows with it. This is the natural rhythm of Agile development applied at the specification level: the boundary is not built all at once, but incrementally, in step with the team's evolving knowledge. What makes this process more tractable than it may appear is AI assistance. Once design decisions are recorded and BDD scenarios are drafted, AI generates test structures and step definition stubs quickly under human review, and implementations against those tests require little effort beyond supervision. The team's attention can therefore remain where it is most needed: understanding requirements clearly enough to design against them.

For feature extensions on existing code, the existing test suite already defines a set of established behavioral statements. New specifications add new statements to that set; AI implements within the extended boundary; and the full suite, old and new tests together, confirms that the extension realizes the new specification without altering the truth value of any existing statement.

The invariant throughout: **specification and design precede generation at every level**. No AI generation begins at any level until the tests that express the specification and design for that level exist and are confirmed red.

Bug fixes follow the same trust boundary logic; see Appendix B.

7.2 Phase 1 — BDD Specification (BDD Layer)

Step 1.1 — Write the feature brief. Before any tool is opened, write a plain-language description of the feature: what user need it addresses, what the observable outcome is when it works, and what the key failure modes are. Two paragraphs is sufficient. This is the input to BDD scenario generation.

Step 1.2 — Generate and review BDD scenarios. Using the feature brief as input, prompt an AI tool to draft a set of Gherkin scenarios covering the happy path, failure paths, and edge cases. Review every scenario critically: - Does each scenario test observable behavior, not implementation detail? - Does each “Given” represent a realistic and complete precondition? - Does each “Then” assert a user-observable outcome? - Are there missing scenarios the AI did not generate?

Revise, add, and delete until the scenario set accurately captures the feature’s behavioral contract. The human owns this artifact — AI drafted it; the human finalized it.

Step 1.3 — Commit the scenarios as a working draft. Commit the .feature files to version control as a working draft. These scenarios will evolve as architecture design and test writing reveal gaps — committing early makes that evolution visible and traceable.

7.3 Phase 2 — Architecture Design

Step 2.1 — Design the system structure. With the BDD scenarios providing a behavioral contract, design the architectural components the feature requires: microservice involvement, new APIs, data flows between services, and repository boundaries. Record architectural decisions as ADRs.

Step 2.2 — Create the implementation skeleton. Translate the architecture design into code structure: packages, interfaces, class signatures, method stubs, API route declarations, repository interface definitions. The skeleton compiles but contains no business logic — every method returns a zero value, a placeholder error, or a not-implemented exception.

The skeleton is the contract between architecture and implementation. It embodies every structural decision in executable form. SBR testing now has concrete targets to specify behavior against.

Step 2.3 — Write design annotations for the generation agent. For each element of the skeleton, write annotations that communicate design intent to the AI generation agent. These are the inputs to the generation phase that tests alone cannot provide: the architectural rationale, interaction patterns, responsibility boundaries, and constraints that shaped the design decisions.

Tests encode *what* the system shall do. Design annotations encode *how* the system shall be structured. They are separate artifacts that must travel separate paths to separate agents — one agent derives tests from the design, another generates implementation from the tests and the annotations. An agent that does both contaminates the falsifiability of the tests.

Current approaches include method-level docstrings describing algorithm intent and caller context; class-level documentation of single responsibility and design pattern; package or service README files covering the component's role, dependencies, and governing ADR; and direct references to architecture diagrams as authoritative constraints.

The specific forms of design annotation may evolve as AI coding tools mature. The separation principle should stay: tests must be authored from the design independently of the implementation. The code generation agent then reads both the design annotations and the tests — but must never modify the tests it implements against without human consent.

7.4 Phase 3 — SBR Test Pyramid Specification (L1–L3) and BDD Wiring

Once the skeleton exists, all specification streams become independent and can proceed in parallel: component tests, unit tests, contract tests, and BDD step definition wiring share no ordering dependency. Different engineers or AI-assisted sessions can tackle them simultaneously. The skeleton is the common substrate that makes parallel specification possible.

Step 3.1 — Write component tests. For each microservice involved in the feature, write component tests that specify the service's expected behavior at its API boundary. Mocks define what external dependencies are expected to receive and return. Structure the test scenario using BDD thinking (Given this API request, When the service processes it, Then this response should be returned), but implement it in the team's standard test framework, not a Gherkin runner.

A component test that passes before any implementation is present is most likely not correctly written — it is not detecting the behavior it claims to verify.

Step 3.2 — Write TDD unit tests. For each method in the skeleton, write a failing unit test that specifies the method's expected behavior. Apply the hypothesis testing discipline: the test is the hypothesis, the red result confirms the hypothesis is falsifiable, and the green result after AI generation confirms the implementation is consistent with it.

Each unit test should specify exactly one behavior. If a test can fail for two different reasons, it needs to be split. The test's precision is the constraint that guides AI generation — imprecise tests produce imprecise implementations.

Step 3.3 — Write contract tests. For each service boundary, write consumer-driven contract tests that specify what each service expects from its providers [19][20]. These are L3 tests — they verify the interface contracts that allow services to be developed and deployed independently.

Step 3.4 — Wire BDD step definitions. With the skeleton in place, wire the step definitions with empty stubs that target the skeleton's concrete method signatures and API contracts. The skeleton gives the step definitions real targets to bind to rather than stubs in a vacuum.

Step 3.5 — Confirm the full trust boundary is red. Run the complete test suite: L1 unit tests, L2 component tests, L3 contract tests, and the BDD Gherkin suite. Confirm every test fails for the right reason: because the behavior does not yet exist, not because of a configuration error or a poorly written test. A test that passes before implementation is present is not detecting the behavior it claims to specify.

Before running the test suite, verify that the RTM is complete for the current scope: every in-scope EARS requirement maps to at least one BDD scenario, and every BDD scenario maps to

pyramid tests at the levels the design requires. A red test suite with RTM gaps is a trust boundary with holes. Generation may proceed for covered requirements, but uncovered ones must be explicitly deferred to a later cycle or explicitly accepted as out of scope for this one.

This is the compliance and cultural timestamp of the SBR workflow. The full trust boundary — spanning the specification, architecture, and test streams — is now established and confirmed red. Every behavioral commitment has been formally specified before a single line of implementation exists. AI generation is authorized to begin.

7.5 Phase 4 — AI Generation

With the full specification and design in place — BDD Gherkin scenarios committed, component tests written, unit tests written, contract tests written, and the implementation skeleton established — the problem space is precisely bounded. This is the moment to hand the work to AI.

AI coding agents are most effective when given the complete picture: the skeleton, the full test suite, the design context, and the freedom to determine their own plan of execution. Constraining AI to one method or one level at a time prevents it from applying its planning capability across the full problem space. A capable AI agent will read the tests, understand the design intent, identify dependencies between components, and choose its own execution order — much as a diligent and competent contracted engineer would.

The specification-first prompt pattern provides the framing. The following is an illustrative example; the specific wording will evolve as AI coding tools mature, but the structural principle should be preserved: full specification and design context must precede the generation request.

“Here is the implementation skeleton, the full test suite, and the relevant design context for this feature. The tests define the behavioral and design contract at every level. Generate implementations that make the tests pass. Do not modify the tests. Raise any ambiguity, impasse, or best practice suggestion before proceeding — do not work around them silently.”

The last instruction is the most important. An AI that silently works around an ambiguity or makes an undocumented design decision is producing invisible drift. An AI that surfaces the ambiguity for human resolution is behaving as a trustworthy collaborator. The human’s role in this phase is not to supervise every step — it is to be available when AI needs judgment that only the human can provide.

When AI suggests changing a test to fit its implementation, treat this as a diagnostic signal worth investigating. The AI may have identified a genuine specification gap, an incomplete design, or an architectural constraint that was not visible in the test alone. These are valuable signals — not defects to be dismissed, and not proposals to be accepted blindly. Understand the cause before deciding.

7.6 Phase 5 — Specification Verification (BDD Layer)

Step 5.1 — Run the full BDD Gherkin suite. With all pyramid levels green, run the full BDD Gherkin suite against the fully assembled system in a staging or production-like environment. This is the grounding proof. A passing suite confirms that the specification is realizable — the system, as assembled, does what the users were promised. A failing suite reveals integration failures that the pyramid levels could not detect.

Step 5.2 — Treat BDD failures as specification signals. When BDD scenarios fail after all pyramid tests pass, the failure is almost never a simple bug in one method. It is a specification gap: an unwarranted implicit assumption in the scenario, a missing precondition, or a service ordering dependency that no pyramid level could detect. Investigate by tracing the failure back through the pipeline to find where the intent was lost.

Step 5.3 — Run smoke tests on deployment. When deploying to any environment, run the curated smoke test subset of the BDD suite. This confirms the system is alive and correctly configured without the overhead of the full suite. A smoke test failure signals a deployment problem; it does not imply a specification failure [25].

Step 5.4 — Refactor under green. With both the pyramid and the BDD suite passing, ask the AI to suggest structural improvements. Accept suggestions only when the full suite remains green after each change. The passing SBR verification chain, pyramid and BDD layer together, is the safety net that makes refactoring safe and AI-assisted refactoring fast.

7.7 The Specification-First Prompt Pattern

The framing of every AI generation prompt determines whether you are practicing SBR or generating code with extra steps. The structural rule is simple: **every prompt must reference a pre-existing, confirmed-red test.**

The examples below are illustrative — they show what should be included and what should be excluded in a specification-first prompt. The optimal phrasing will evolve as AI coding tools mature; the structural principle will not.

What a specification-first prompt must include: - The failing test or tests — the behavioral and design contract AI is generating against - The implementation skeleton or relevant code context — so AI understands the design decisions already made - An explicit instruction not to modify the tests — reinforcing that the tests are the specification, not a suggestion - An instruction to surface ambiguity or design conflicts rather than working around them silently

What a specification-first prompt must not include: - A request to write tests alongside the implementation — this is generate-then-verify - A vague feature description without a test — this gives AI no ground truth - Permission to modify existing tests to accommodate the implementation

The following contrasting examples illustrate the difference. In the correct example, the test is the input that constrains the generation. In the incorrect example, AI is asked to define both the specification and the implementation simultaneously:

Correct — specification-first:

I am implementing the `validatePasswordReset()` method. Here is my failing unit test:

[paste test code]

The test asserts that `validatePasswordReset()` with an expired token:

- Returns a failure result with error code `TOKEN_EXPIRED`
- Does not update the user's password
- Does not invalidate the token (expired tokens are self-invalidating)

Implement `validatePasswordReset()` to make this test pass.
 Do not modify the test. Do not add behavior not specified in the test.
 Raise any ambiguity or design conflict before proceeding.

Incorrect — generate-then-verify:

Implement a password reset validation function and write tests for it.
 The second prompt will produce code and tests that agree with each other. They may have nothing to do with what was actually required. The first prompt produces implementation constrained by a human-specified behavioral and design claim — the only kind of constraint that makes AI generation trustworthy.

8 Summary: The SBR Framework at a Glance

The SBR Test Pyramid — implementation verification (Table 5)

Table 5. Trust Boundary Construction — The SBR Test Pyramid				
Level	What It Tests	Who Specifies	Who Generates	Correctness Claim
L3 — Contract	Service boundary behavior	Human (consumer-driven)	AI (adapter code)	Interfaces honor their contracts
L2 — Component	Microservice internal logic	Human (BDD thinking, standard framework)	AI (service wiring)	Service assembly is correct
L1 — Unit	Method behavioral contract	Human (TDD tests)	AI (method implementation)	Method honors its promise to callers

The BDD Specification Layer — specification verification (Table 6)

Table 6. Trust Boundary Grounding — The BDD Specification Layer				
Artifact	What It Tests	Who Specifies	When It Runs	Grounding Claim
Full BDD Gherkin Suite	Complete user journeys	Human (from EARS requirements)	End of sprint / before release	System realizes the specification
Smoke Tests	Critical deployment paths	Human (curated BDD subset)	Every deployment	System is alive and correctly configured

The Requirements Traceability Matrix — boundary completeness (Table 7)

Table 7. Trust Boundary Completeness — The Requirements Traceability Matrix

Instrument	What It Verifies	When It Is Checked
RTM	Every in-scope requirement maps to BDD scenarios and pyramid tests at the required levels	Before generation begins (Step 3.5 in Phase 4, Section 7.4)

The trust boundary: The trust boundary is the central concept of SBR. It represents the requirements and design decisions of the system in executable, verifiable form. The SBR practice — through the test pyramid, the BDD grounding proof, and the RTM — ensures the boundary is both effective and complete. A well-maintained trust boundary is the foundation that makes AI coding automation trustable.

9 Appendix A — Non-functional requirements Testing: What SBR Can and Cannot Cover

Non-functional requirements (NFRs) are the system qualities that describe how a system performs rather than what it does. Examples include performance, security, scalability, accessibility, maintainability, and compliance. SBR addresses NFRs unevenly, and understanding that boundary is important for building a complete quality strategy.

9.1 NFRs Directly Testable Within SBR

Performance and response time. BDD scenarios can include timing assertions — “When a search query is submitted, the system shall return results within 200ms” maps directly to a Gherkin scenario in the BDD specification layer with a timing check. TDD unit benchmarks can verify performance at the method level. These are behavioral NFRs that describe how the system behaves under normal conditions, and they fall well within SBR’s scope.

Security behaviors. Authentication failure modes, authorization boundary enforcement, input validation rules, and error response handling are all expressible in Given/When/Then at the SBR test pyramid and BDD specification layer levels. “When an unauthenticated request is received, the system shall return 401 with no data in the response body” is an L2 component test. “When a user attempts to access another user’s data, the system shall return 403” is a BDD Gherkin scenario. Security behaviors are among the most important NFRs to specify in SBR precisely because AI generation can produce plausible but insecure defaults if security requirements are not expressed as explicit, failing tests.

Error handling and resilience. Circuit breaker behavior, retry logic, graceful degradation under dependency failure, and timeout handling are all specifiable as TDD unit tests (L1) or component tests (L2). These NFRs are particularly well-suited to SBR because they are event-driven — they describe the system’s response to specific undesirable conditions, which is exactly the EARS unwanted behavior pattern.

9.2 NFRs Requiring Specialized Tooling Alongside SBR

Scalability and load. BDD provides the acceptance criteria — “the system shall support 1,000 concurrent users with response times below 500ms” — but the load generation requires specialized tools (k6, Locust, JMeter, Gatling). SBR defines the pass/fail criterion; the load testing framework provides the execution environment. Both are necessary.

Accessibility. BDD scenarios can express accessibility requirements — “When a screen reader user navigates to the login form, all input fields shall have accessible labels” — but automated accessibility verification tools (axe, Lighthouse, Pa11y) are needed to check the rendered DOM against WCAG standards. SBR and accessibility tooling are complementary.

9.3 *NFRs Outside SBR’s Scope*

Maintainability and code quality. These are structural NFRs, not behavioral ones. Static analysis tools, coupling metrics, cyclomatic complexity thresholds, and test coverage requirements are enforced by architecture fitness functions [26]. SBR does not cover these — fitness functions do.

Usability. Usability requires human judgment about the quality of user experience. It cannot be automated at a meaningful level of fidelity. User research, usability testing, and A/B testing are the appropriate verification mechanisms.

Legal and regulatory compliance. Compliance is verified through audit, documentation review, and process assessment — not automated testing. SBR test records may form part of a compliance evidence package, but they cannot substitute for the audit process itself.

9.4 *The Complete NFR Coverage Strategy*

A complete NFR coverage strategy requires all four mechanisms working together:

- **SBR** covers behavioral NFRs across the SBR test pyramid and BDD specification layer — performance, security, error handling, resilience
- **Specialized tooling** (load testing, accessibility scanners) covers performance-at-scale and accessibility
- **Architecture fitness functions** cover structural NFRs — maintainability, coupling, coverage thresholds
- **Audit and process assessment** covers compliance

10 **Appendix B — Bug Fixes in the SBR Framework**

10.1 *What a Bug Means in SBR Terms*

In the SBR framework, a bug — whether discovered in testing or after delivery — is evidence of a gap in the trust boundary. The system exhibited behavior that was not correctly specified — either because the specification was incomplete (the behavior was never defined), inconsistent (the behavior was defined contradictorily), or because the implementation did not satisfy a specification that existed. In all three cases, the trust boundary failed to catch the problem: a test that should have caught the incorrect behavior either did not exist or was not precise enough to detect it.

This framing has an important consequence: before any code is changed, the nature of the gap must be understood. Fixing the code without understanding where the trust boundary failed produces a local correction that may not address the underlying cause — and risks introducing

new failures in adjacent behavior. The SBR approach to bug fixing is therefore root cause first, tests second, implementation third — in that order, without exception.

10.2 Step 1 — Root Cause Analysis

The first step is to ask AI to analyze the failing behavior and identify the root cause. This is diagnostic work, not implementation work. The prompt should be framed accordingly:

“Here is the observed bug: [description of the incorrect behavior]. Here is the relevant code: [paste]. Here is the existing test suite: [paste]. Analyze the root cause without modifying any code. Map the root cause to one of three categories: missing requirements, architecture design gap, or implementation design gap. Explain your reasoning.”

The three categories determine where the fix must be applied and at which level of the SBR test pyramid:

Missing requirements — the behavior was never specified in the BDD scenarios or EARS requirements. The system was never told what to do in this situation. The gap is at the specification layer — the trust boundary was never drawn here at all.

Architecture design gap — the behavior was intended but the component structure, service boundary, or interface contract does not support it correctly. The gap is at L2 or L3 — the trust boundary was drawn at the wrong level of the architecture.

Implementation design gap — the behavior was specified and the architecture supports it, but the method implementation is incorrect. The gap is at L1 — the trust boundary was drawn correctly but the implementation crossed it without detection because the test was insufficiently precise.

10.3 Step 2 — Write Failing Tests

With the root cause mapped to a category, write the tests that specify the correct behavior at the appropriate level. The tests must fail against the current codebase — this confirms that they are genuinely verifying the missing behavior, not passing accidentally.

Do not touch the implementation before the tests are written and confirmed red. A developer who fixes the code first and writes tests afterward is repeating generate-then-verify in a maintenance context. The tests must precede the fix, for precisely the same reason they must precede new feature implementation.

The level at which tests are written depends on the root cause category:

- **Missing requirements** — write a new BDD Gherkin scenario at the specification layer, then propagate down: write component tests (L2) and unit tests (L1) as needed to cover the specific behavior gap at each level
- **Architecture design gap** — write failing component tests (L2) or contract tests (L3) that expose the incorrect service behavior
- **Implementation design gap** — write a failing unit test (L1) that precisely specifies the correct method behavior

10.4 Step 3 — Implement the Fix

With failing tests in place, provide AI with the complete context: the existing codebase, the existing passing test suite, the new failing tests, and the root cause analysis.

“Here is the root cause analysis: [analysis]. Here are the new failing tests: [paste]. The existing test suite must continue to pass. Implement the minimum fix to make the new tests pass without modifying any existing tests. Raise any ambiguity or design conflict before proceeding — do not work around them silently.”

The constraint that existing tests must continue to pass is the trust boundary enforcing regression safety. A fix that breaks existing tests has introduced a regression — the trust boundary has been violated. AI should flag this rather than silently modify existing tests to accommodate the fix.

10.5 Step 4 — Verify and Investigate Further

When the new tests pass and the existing suite remains green, the fix is structurally sound. The trust boundary has been extended to cover the previously missing behavior. Deploy the fix and verify it in the live system — confirm that the reported bug no longer manifests in the deployed environment and that no regressions are visible to users. This concludes the bug fix.

But the SBR discipline requires one additional step: **review the root cause for hidden problems**. A bug is rarely isolated. If a missing requirement allowed one incorrect behavior, it may have allowed others in adjacent scenarios. If an architecture design gap produced one incorrect service interaction, adjacent interactions may be similarly affected.

The developer should ask:

- Given the root cause just identified, what other behaviors in the system might be similarly affected?
- Are there adjacent BDD scenarios or component tests that should be strengthened to cover the now-visible class of gap?
- Does this root cause suggest a systematic weakness — a category of requirement that was undertested from the start?

This review may produce additional failing tests, which drive additional fixes through the same cycle. The trust boundary expands incrementally with each iteration, becoming more complete with each bug fixed.

10.6 When New Tests Pass But the Bug Persists

If the new tests pass but the reported bug remains visible in the running system, the root cause analysis was incorrect. The tests verify behavior adjacent to — but not identical to — the actual failure point. The fix addressed the wrong gap.

This is valuable information. The developer now knows what the bug is *not* caused by. Restart the analysis with this knowledge explicitly included:

“The previous root cause analysis identified [root cause]. Tests were written and pass for that behavior. The bug persists. Given that [previous root cause] has been ruled out, analyze the remaining possibilities.”

Each iteration of analysis → test → implement → verify narrows the search space. The cycle is the scientific method applied to debugging: each experiment either confirms or falsifies a hypothesis about the root cause. The failing tests produced in each iteration are not wasted — they permanently strengthen the trust boundary, even when they did not locate the bug.

10.7 The Bug Fix Cycle at a Glance

- | | |
|-------------------------------------|--|
| 1. AI analyzes root causes | ← diagnostic only, no code changes |
| 2. Root causes mapped to categories | ← missing requirements / architecture gap / implementation gap |
| 3. Failing tests written | ← at the level corresponding to the root causes |
| 4. Tests confirmed red | ← verifies tests are genuine, not accidentally passing |
| 5. AI implements the fix | ← within trust boundary, existing tests must stay green |
| 6. All tests green | ← bug is fixed and regression safety confirmed |
| 7. Verify fix in the live system | ← confirm the bug is resolved in the deployed environment |
| 8. Review for hidden problems | ← extend the trust boundary where adjacent gaps exist |

If tests pass but bug persists: root cause was incorrect → restart Step 1 with accumulated knowledge.

The invariant: Tests specify the correct behavior before any code is modified. The trust boundary is never repaired by changing code without first changing the tests.

11 Appendix C — Formal Grounding of the Trust Boundary

This appendix provides a semi-formal treatment of the theoretical foundations underlying the trust boundary concept introduced in Section 5. It covers five areas: the mapping of the trust boundary to a formal axiomatic system (C.1); what a passing test suite establishes, including its relationship to Gödel’s incompleteness theorem (C.2); the taxonomy of incompleteness from axiomatic and operational perspectives (C.3); the grounding of the SBR test pyramid in Gurevich’s Abstract State Machine theory (C.4); and the formal basis of the Given/When/Then structure in temporal logic (C.5). C.2 contains an optional discussion intended for readers familiar with Gödel’s incompleteness theorem or Russell’s paradox; the analysis may also serve as a theoretical foundation for applying the trust boundary concept to other domains of AI automation.

11.1 C.1 — The Trust Boundary as a Formal Axiomatic System

The trust boundary concept has a precise structural analogue in formal logic.

A **formal axiomatic system** consists of a set of axioms — statements accepted as true without proof — and a set of inference rules that allow new statements (theorems) to be derived from the axioms. A statement is *provable* within the system if it can be derived from the axioms using the inference rules. A statement that cannot be derived is outside the provability boundary of the system — it is neither confirmed nor denied by the system; it is simply not addressed.

In the SBR trust boundary:

- **Tests are axioms.** Each test is a formal behavioral statement: given this precondition, when this action occurs, then this outcome shall hold. The statement is written and approved by a human before any implementation exists.
- **Tests are also theorems relative to the design.** Tests are derived from design decisions upstream — they are theorems of the design system and axioms of the implementation system. A test that does not correctly reflect the design is a flawed axiom, corrupting the

formal system it anchors. This is why human review of tests before generation is not overhead — it is the step that establishes the integrity of the axiom set.

11.2 C.2 — What the Trust Boundary Proves

An implementation that passes all tests exhibits behaviors consistent with the full axiom set — the test suite cannot falsify them. But passing tests do not uniquely determine the implementation. Multiple different implementations may satisfy the same set of tests; what the tests do is bound the space of acceptable implementations. Because every software system is a state machine, specifically a Turing machine, a passing implementation is therefore a **candidate theorem**: consistent with the formal system defined by the test suite, but not necessarily the unique consequence of the axioms. The candidate theorem becomes more constrained — closer to uniquely determined — as more tests are added. The trust boundary is the provability boundary of this system: the region of behavior within which correctness has been established as a candidate theorem. Behavior outside this region is neither proven correct nor proven incorrect — it is outside the system entirely.

Readers familiar with Gödel’s incompleteness theorem or Russell’s paradox may wonder how the trust boundary relates to the fundamental limits of formal systems. The following paragraphs address that question. The analysis also informs how the trust boundary concept extends beyond software development to other AI automation domains. Readers not interested in this connection may proceed directly to C.3.

Gödel’s first incompleteness theorem establishes that any consistent formal system sufficiently expressive to encode arithmetic contains true statements that cannot be proven within the system [27]. A passing test suite has the same property: it establishes correctness within the region the tests cover, not beyond it.

The geometry of the parallel postulate offers a concrete picture. Euclid’s first four postulates are consistent but incomplete: without the fifth postulate, the angle sum of a triangle is provably at most 180° but cannot be proven equal to exactly 180° . Both Euclidean geometry and hyperbolic geometry satisfy the first four postulates — so neither the Euclidean nor the hyperbolic answer is derivable from those axioms alone. The unprovable statement is not false; it is simply outside what the axiom set can establish.

A system behavior the test suite does not cover is in exactly that position: neither proven correct nor proven incorrect — outside the trust boundary entirely.

The similarity ends there, however. In formal logic, incompleteness is inherent — any consistent system powerful enough to encode arithmetic cannot be made complete. In software terms, this means there is no single grand master system that covers all possible states of an application domain. A software specification defines a bounded set of behaviors by design, not an exhaustive account of every possible system state. A state not covered by the specification is outside the contract, not a deficiency. The trust boundary’s incompleteness is therefore extrinsic and eliminable on its own terms: every state the specification intends to cover can be tested, and every gap within that boundary reflects an unwritten test, not an unwritable one.

11.3 C.3 — Three Sources of Incompleteness

The analysis in C.2 establishes one key result: the trust boundary is always incomplete. This is not a failure of the SBR framework — it is a fundamental property of any formal specification system applied to a real-world software system.

Incompleteness can be examined from two perspectives. The first is the **axiomatic perspective** — the formal logic view that identifies the axiomatic cause of each gap, explaining why a given behavior falls outside the trust boundary. The three conditions introduced in Section 5 provide this view:

- **Undetected incorrectness** corresponds to a missing axiom: a statement that should have been in the formal system but was not included. The implementation is incorrect in this region, but the system has no means to derive that incorrectness from its existing axioms.
- **Undefined behavior** corresponds to a statement that is genuinely outside the intended scope of the formal system. No axiom is missing — the system was never designed to address this case. The behavior exists but the formal system is silent about it.
- **Miss-specified behavior** corresponds to a flawed axiom: a statement present in the formal system that does not correctly represent the intended specification. The boundary is drawn, but in the wrong place.

The second is the **operational perspective** — the practice view that identifies what category of specification gap produced the incompleteness, explaining where the gap came from in the development process:

- **Intentional incompleteness** — behaviors deliberately left unspecified because they are out of scope, not yet prioritized, or handled by a separate component. The boundary does not extend here, and that is by design.
- **Accidental incompleteness** — behaviors that should have been specified but were missed. These are the classical bugs-in-waiting: correct behavior was never declared, so incorrect behavior cannot be detected. The remedy is to extend the trust boundary by writing the missing tests.
- **Structural incompleteness** — behaviors that arise from the composition of correctly specified components in ways that no individual test covers. This is the gap that the BDD grounding proof addresses: components that are individually correct at L1, L2, and L3 may compose incorrectly at the system level. The BDD suite is the formal instrument that detects structural incompleteness. The timezone example in Section 6 illustrates this precisely: each component’s specification was complete and correct at every pyramid level, yet the assembled system failed because the composition of two correct components produced wrong behavior under real deployment conditions — a gap that only the BDD grounding proof could detect.

The distinction between accidental and structural incompleteness explains why a pyramid of green tests is necessary but not sufficient for a working system. Green pyramid tests establish that each refinement step in the chain is locally correct. The BDD grounding proof establishes that the full chain — from ground model to execution — is globally correct. Both are required.

11.4 C.4 — Gurevich’s Abstract State Machine Theory and the Refinement Chain

The SBR pyramid is not an arbitrary layer structure. Each level corresponds to a level of abstraction at which system behavior can be formally specified, and the levels form a refinement chain formalized in Gurevich’s Abstract State Machine (ASM) theory. An Abstract State

Machine is a formal computational model that describes the behavior of a system as a sequence of state transitions governed by explicit rules [10][11].

ASM theory makes four formal claims that directly support SBR's structure:

- **Ground model theorem** — every algorithm can be fully and precisely described as an ASM at any level of abstraction. BDD is the ground model: the most abstract, complete specification of system behavior before any structural decisions exist.
- **Refinement** — any two abstraction levels can be formally related by a refinement proof: the lower level faithfully implements the higher without losing behavioral information. Each level of the test pyramid is a formal refinement step [28].
- **Level independence** — each abstraction level is independently specifiable and independently complete. Pyramid levels are formally distinct: a method-level specification is not an approximation of a component-level one, nor vice versa.
- **Completeness** — every sequential algorithm can be captured by an ASM (the sequential ASM theorem [10]). The SBR refinement chain is therefore not an approximation — it is formally complete at every level, provided each refinement step is correctly specified.

The SBR pyramid maps onto the ASM refinement chain as follows (Figure 4).

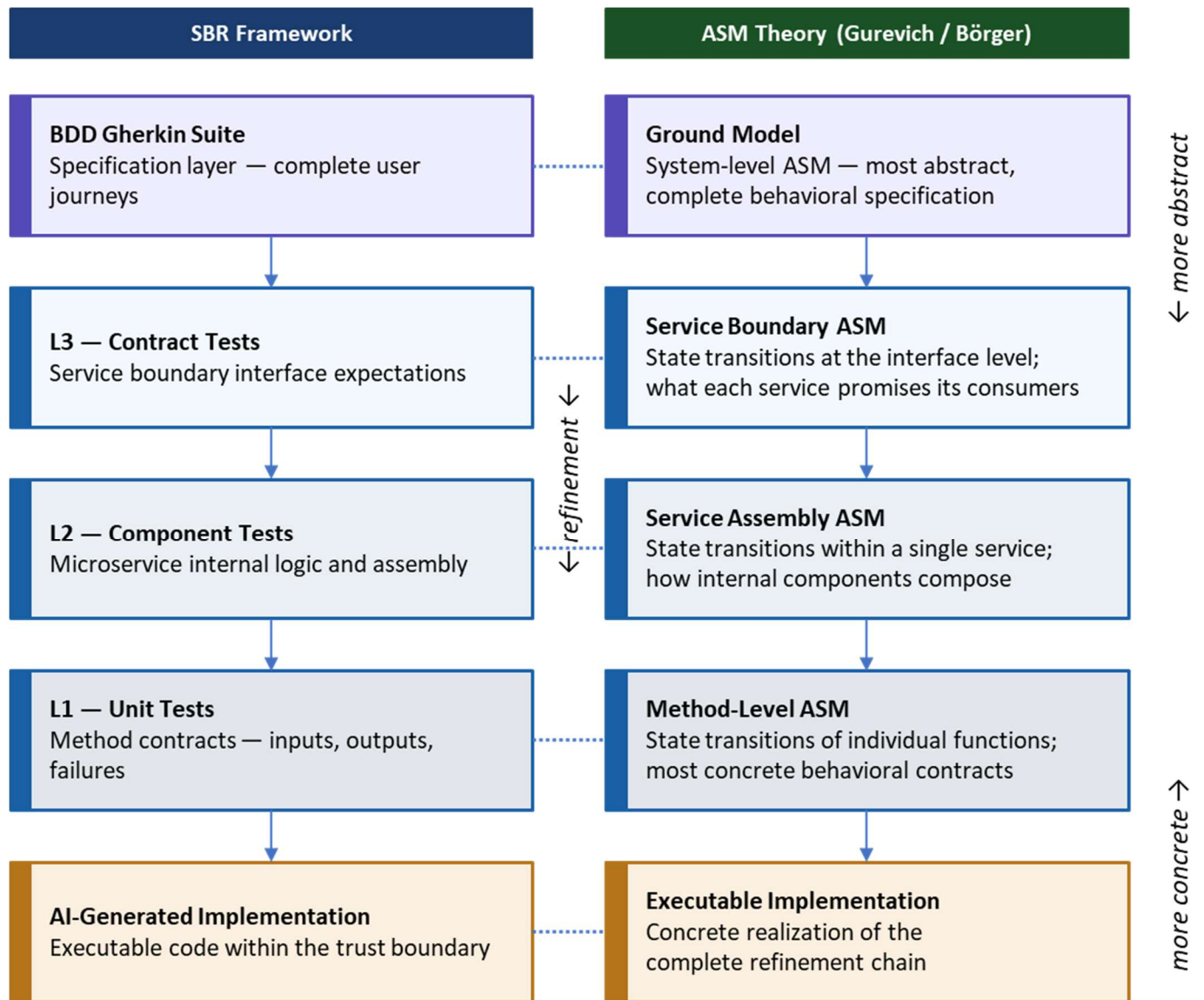


Figure 4. Correspondance between SBR test layers and ASM abstraction layers

Each refinement step is a formal proof: the lower level faithfully implements the higher level without losing behavior information [28]. A gap in the trust boundary at any level is a missing refinement step — a claim about the system’s behavior that has not been formally established.

When all levels pass — L1, L2, L3, and the BDD Gherkin suite — the full refinement chain is complete. The system’s behavior has been formally established from the ground model down to executable implementation. This is what the paper calls grounding: not a metaphor, but a formal property of the refinement chain.

11.5 C.5 — Temporal Logic and the Given/When/Then Structure

ASM theory is built on a variation of **temporal logic** — the formal language for reasoning about how system state changes over time [29][30][31].

A temporal logic formula expresses properties of state sequences. The most relevant operators for understanding tests as formal specifications are:

- **Precondition (Given):** a predicate that must hold before an action occurs.
- **Action (When):** a transition that changes the system state.
- **Postcondition (Then):** a predicate that must hold after the transition.

This is precisely the structure of a Given/When/Then scenario in Gherkin, and of a unit test's Arrange/Act/Assert pattern. The structural parallel is not accidental. Both BDD's Gherkin notation and the xUnit test framework's assertion model are informal encodings of the same temporal logic statement: *if the system is in state S and action A occurs, then the system transitions to state S'.*

Formally, a test at any level of the SBR pyramid is a **safety property** in temporal logic terms — a statement of the form “bad state S' is never reached from state S via action A.” A passing test suite is therefore a set of verified safety properties over the system's state space. The trust boundary is the region of the state space covered by these verified safety properties. Liveness properties are addressed separately.²

12 Appendix D — How the SBR Framework Addresses the Unresolved Problems of the Waterfall/Agile Paper

This appendix presents the assessed resolution of the eight unresolved problems identified in the companion paper Waterfall and Agile in the Era of AI Coding [1], organized by problem nature.

12.1 Group 1 — Specification Fidelity Problems

SBR directly solves all problems in this group. Specification fidelity is the framework's core purpose.

W3 — Agile's JBGE and code-as-documentation practices conflict with AI's need for precise behavioral specification. *Directly solved.* BDD scenarios resolve the tension by making behavioral specification and executable documentation the same artifact — produced before code exists.

W4 — Specification quality has no verification mechanism. *Directly solved.* A specification that cannot be expressed as a failing test is not complete. The executable specification is the verification mechanism. The Requirements Traceability Matrix (RTM), introduced in Section 5, operationalizes this claim: it provides an explicit traceability map from each in-scope EARS requirement to its BDD scenarios and pyramid tests at every required level. RTM completeness is the structural evidence that the trust boundary covers every requirement currently in scope.

W5 — Formal acceptance criteria are not operationalized. *Directly solved.* The EARS → Gherkin pipeline operationalizes acceptance criteria as runnable BDD Gherkin scenarios.

² Liveness properties — statements of the form “good state S' is eventually reached” — require a different encoding than safety properties. In practice, TDD addresses liveness through bounded liveness tests: the test waits up to a defined timeout for the expected state to be reached, then asserts. The timeout window is typically a non-functional requirement, defined alongside the behavioral specification. This is the standard and sufficient treatment for liveness in production software systems.

W8 — The wrong thing can be built faster than ever. *Directly solved.* BDD Gherkin scenarios surface the wrong-thing signal before delivery. A failing BDD suite stops wrong-thing accumulation.

12.2 Group 2 — Process Cadence Problems

SBR provides automated gates that scale with AI velocity. Human process redesign remains an open question.

W7 — Technical review cadence is unspecified. *Directly solved.* The SBR test pyramid running on every commit is the automated review mechanism. Human review shifts to specification and test quality.

W2 — Sprint mechanics become incoherent under AI velocity. *Partially solved.* Specification completeness and test coverage replace story points as sprint metrics. The methodology question of sprint sizing remains open.

W6 — Technical debt accumulates faster than sprints can absorb it. *Partially solved.* L2 component tests and fitness functions make debt visible earlier. Sprint prioritization of debt repayment remains a methodology question.

12.3 Group 3 — Human Uncertainty Problems

SBR reduces exposure but cannot eliminate the underlying uncertainty.

W1 — Requirements uncertainty is not resolved by AI. *Partially solved.* BDD Gherkin scenarios surface the specification-to-reality mismatch earlier and cheaper. Requirements uncertainty is reduced; it is not eliminated. The iterative Agile feedback loop remains necessary.

12.4 Summary

The resolutions (Table 8) marked as partially solved represent the current frontier of SBR's application. The analyses presented here are preliminary — they identify the direction in which the framework is useful and the boundaries where it reaches its limits, but they do not constitute validated practice. Further research, empirical case studies, and tooling development are needed to mature these resolutions into established practice.

Table 8. SBR addresses Unresolved Problems in the Waterfall/Agile Paper

Group	Problem	Resolution
Specification Fidelity	W3, W4, W5, W8	[solved] Directly solved (4 of 4)
Process Cadence	W7	[solved] Directly solved
	W2, W6	[partial] Partially solved
Human Uncertainty	W1	[partial] Partially solved

The SBR framework is, at its core, a **specification fidelity mechanism** — its purpose is to close the gap between what was specified and what was implemented. All four specification fidelity problems are directly solved. The two process cadence problems are partially addressed: SBR contributes automated gates and a specification-completeness standard, but the methodology questions around sprint sizing and debt prioritization are not resolved. The human uncertainty problem is partially addressed: BDD scenarios surface the specification-to-reality mismatch earlier, but requirements uncertainty itself is not eliminated. The partially-solved resolutions

represent the current frontier of SBR’s application. They identify the direction in which the framework is useful and the boundaries where it reaches its limits, but they do not constitute validated practice. Further research, empirical case studies, and tooling development are needed.

Research questions that SBR practice surfaces — distinct from the W# problems inherited from the companion paper — are collected in Appendix E.

13 Appendix E — Open Research Questions (R# Series)

This appendix collects research questions that SBR practice surfaces — problems that SBR reasoning identifies as important but that are not yet resolved by field experience or empirical evidence. They are distinct from the W# series, which are problems inherited from the waterfall/agile companion paper and assessed for resolution in Appendix D. The R# series records questions that applying SBR reveals, and hands them forward to future research.

R1 — Negative case completeness has no principled stopping criterion.

Positive case specification is bounded by requirements: every EARS functional requirement maps to one or more BDD scenarios, and RTM completeness provides a stopping criterion. Negative case specification does not have an equivalent bound. Known failure modes — those derivable from EARS unwanted behavior requirements — are specifiable upfront and appear in the RTM. Implementation-specific failure modes are unknowable before the implementation exists; they are discovered through operation and addressed iteratively via the Appendix B bug fix cycle. No mechanism exists to verify that all material failure modes have been specified before generation begins. The practical boundary between “specified enough” and “undertested” is a judgment call that the current SBR framework does not resolve. Empirical research on negative case coverage strategies, and tooling that can surface likely failure modes from design artifacts before implementation begins, are needed to mature this aspect of SBR practice.

R2 — Design-anchored test traceability requires a second RTM anchor.

The dual-anchor RTM model — tracing tests to either an EARS requirement or an ADR — is described in Section 5. The model is analytically coherent but has not been validated in practice. How ADRs are created and maintained as governance artifacts is a separate topic outside the scope of this paper. Field experience is needed on three fronts: whether design-anchored tests appear frequently enough in practice to warrant a formal RTM extension; whether ADR anchoring is the right traceability mechanism for them; and whether flagging unanchored tests as explicitly unclassified works as a discipline in real teams. Answers to all three are prerequisites for maturing the dual-anchor model into established practice.

14 References

- [1] Zhou, Yixiong. *Waterfall and Agile in the Era of AI Coding*. Companion document, 2026.
- [2] Beck, Kent. *Test-Driven Development: By Example*. Addison-Wesley, 2002.
- [3] North, Dan. “Introducing BDD.” *Better Software Magazine*, March 2006.
<https://dannorth.net/introducing-bdd/>
- [4] Foojay.io. “AI-Driven Testing Best Practices.” Foojay — Friends of OpenJDK, May 26, 2025. <https://foojay.io/today/ai-driven-testing-best-practices/>

- [5] Nagappan, Nachiappan, E. Michael Maximilien, Thirumalesh Bhat, and Laurie Williams. “Realizing Quality Improvement Through Test Driven Development: Results and Experiences of Four Industrial Teams.” *Empirical Software Engineering* 13:3 (2008), 289–302.
- [6] Zhou, Yixiong. Personal communication. Senior software engineering practitioner with 20+ years of experience. 2026. “*The overhead of rigorous TDD practice typically ranges from 20% to 50% of initial implementation cost. Human engineers writing code without pre-written tests are not being negligent — they are reviewing and interpreting the specification during implementation, and post-hoc tests capture the design decisions made in that process. AI coding removes this implicit review entirely, fundamentally changing the risk profile of skipping pre-written tests.*”
- [7] Inflectra Corporation. “Operationalizing BDD Scenarios Through Generative AI.” EuroSTAR Conference, May 2024.
<https://conference.eurostarsoftwaretesting.com/operationalizing-bdd-scenarios-through-generative-ai/>
- [8] Shah, Nat. “TDD and BDD in the Age of AI: Why AI Agents Demand 100% More Test-First Development.” natshah.com, January 3, 2026. <https://natshah.com/blog/tdd-bdd-age-ai-why-ai-agents-demand-100-more-test-first-development>
- [9] Qodo. “2025 State of AI Code Quality.” Qodo Research, June 2025.
<https://www.qodo.ai/reports/state-of-ai-code-quality/>
- [10] Gurevich, Yuri. “Sequential Abstract State Machines Capture Sequential Algorithms.” *ACM Transactions on Computational Logic* 1:1 (July 2000), 77–111.
<https://dl.acm.org/doi/10.1145/343369.343384>
- [11] Börger, Egon and Robert Stärk. *Abstract State Machines: A Method for High-Level System Design and Analysis*. Springer-Verlag, 2003.
- [12] Beck, Kent. “TDD Isn’t Design.” *Tidy First? (Substack)*, December 7, 2023.
<https://tidyfirst.substack.com/p/tdd-isnt-design>
- [13] Beck, Kent. *Extreme Programming Explained: Embrace Change*, 2nd ed. Addison-Wesley, 2004.
- [14] Agile Alliance. “Behavior-Driven Development (BDD).” Agile Alliance Glossary.
<https://agilealliance.org/glossary/bdd/>
- [15] Cohn, Mike. *Succeeding with Agile: Software Development Using Scrum*. Addison-Wesley Professional, 2009.
- [16] Wikipedia. “Test-Driven Development — Acceptance Test-Driven Development.”
https://en.wikipedia.org/wiki/Test-driven_development
- [17] Agile Alliance. “Acceptance Test-Driven Development (ATDD).” Agile Alliance Glossary.
<https://agilealliance.org/glossary/atdd/>
- [18] Fowler, Martin. “SubcutaneousTest.” Martin Fowler’s Bliki, February 14, 2011.
<https://martinfowler.com/bliki/SubcutaneousTest.html>
- [19] Robinson, Ian. “Consumer-Driven Contracts: A Service Evolution Pattern.” ThoughtWorks, 2006. <https://martinfowler.com/articles/consumerDrivenContracts.html>

- [20] Pact Foundation. “Pact — Consumer-Driven Contract Testing Framework.” <https://pact.io/>
- [21] Popper, Karl. *The Logic of Scientific Discovery*. Hutchinson, 1959.
- [22] Microsoft Engineering Fundamentals Playbook. “Smoke Testing.” Microsoft GitHub, 2024. <https://microsoft.github.io/code-with-engineering-playbook/automated-testing/smoke-testing/>
- [23] Mavin, Alistair, Philip Wilkinson, Adrian Harwood, and Mark Novak. “Easy Approach to Requirements Syntax (EARS).” *Proceedings of the 17th IEEE International Requirements Engineering Conference (RE 2009)*, Atlanta, GA, August 31–September 4, 2009, pp. 317–322. <https://doi.org/10.1109/RE.2009.9>
- [24] Mavin, Alistair, Philip Wilkinson, Sarah Gregory, and Eero Uusitalo. “Ten Years of EARS.” *IEEE Software* 36:5 (September 2019), 10–14. <https://doi.org/10.1109/MS.2019.2921164>
- [25] TestQuality. “How to Integrate Gherkin Testing in Your CI/CD Pipeline.” TestQuality Blog, October 21, 2025. <https://testquality.com/how-to-integrate-gherkin-testing-in-your-ci-cd-pipeline/>
- [26] Ford, Neal, Rebecca Parsons, and Patrick Kua. *Building Evolutionary Architectures: Automated Software Governance*. 2nd ed. O’Reilly Media, 2022.
- [27] Gödel, Kurt. “Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I.” *Monatshefte für Mathematik und Physik* 38 (1931), 173–198. English translation: “On Formally Undecidable Propositions of Principia Mathematica and Related Systems.” Basic Books, 1962.
- [28] Börger, Egon. “The ASM Refinement Method.” *Formal Aspects of Computing* 15:2–3 (November 2003), 237–257. <https://doi.org/10.1007/s00165-003-0012-7>
- [29] Pnueli, Amir. “The Temporal Logic of Programs.” *Proceedings of the 18th Annual Symposium on Foundations of Computer Science (FOCS 1977)*, 46–57. <https://doi.org/10.1109/SFCS.1977.32>
- [30] Clarke, Edmund M., Orna Grumberg, and Doron A. Peled. *Model Checking*. MIT Press, 1999.
- [31] Lamport, Leslie. “Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers.” Addison-Wesley, 2002. (*TLA+ is a practical specification language built on temporal logic, widely used in industry for verifying distributed systems.*)

15 Acknowledgements

The author thanks three reviewers whose comments materially improved this paper.

Steven Offen, formerly Chief Technology Officer and Software Architect at MIRO Technologies and Tapestry Solutions, reviewed both papers. His technical depth and architectural experience shaped the verification and governance arguments in important ways. His comments on this paper produced several specific improvements: his observation that TDD’s cognitive inversion — from “what do I want this code to do” to “how will I know when it does not” — is genuinely unnatural for most practitioners led to the restructured opening of Section 2; his identification of the trust boundary diagnostic paragraph as the paper’s strongest passage prompted its elevation into the Preface; his challenge to the “ground model” framing as an assertion rather than a design

principle led to its removal; and his precise readings of the undefined behavior definition, the BDD suite failure bullet list, and the grounding claim each produced more accurate formulations.

Xinghua Zheng, Chief Engineer, NIWC PAC, reviewed both papers in this series and provided comprehensive structured feedback. Her observations on this paper led to several specific improvements: her comment on adoption complexity prompted the Section 7 overview paragraph connecting the trust boundary to Agile’s iterative cadence; her observation that the framework assumes good tests without explaining how to assess them led to the introduction of the Requirements Traceability Matrix and the Boundary Completeness Section 5.6; her comment that AI’s role felt narrowly defined as a code generator produced the revised summary invariant now centered on the trust boundary as the foundation of trustable AI coding automation. Her cross-paper observations on boundaries, the unifying principle, and audience produced the orientation paragraph now appearing in each paper’s preface.

Franklin Zhou, Software Tech Lead and Architect at Visa Inc., reviewed this paper. His observation that the closing sentence of the TDD subsection (Section 5.6) — asserting high coverage as a side effect without explaining what else the claim implied — was ambiguous and unsupported led to the softened opening qualifier, the clarification of “high code coverage” as the precise term, and the footnote acknowledging that rigorous specification-first discipline is a minority practice in the field. His identification of the negative case specification problem — that failure modes are often implementation-specific and that no principled stopping criterion exists — drove the cultural transition paragraph added to Section 3.1, the negative case treatments in the BDD section and the RTM section (Section 5.6), and the establishment of the R# open research question series in Appendix E, with R1 addressing negative case completeness and R2 addressing design-anchored test traceability. His observation that the trust boundary concept relies on too many acronyms held simultaneously in the reader’s mind, and that a diagram is needed to make the concept tractable, produced the pipeline diagram appearing in the AI Generation section (Section 5.4), showing the full specification-to-verification chain with the trust boundary bracketed within it.

Their collective feedback helped transform early drafts into clearer, more concise, and more readable papers.

This document reflects analysis and practice current as of early 2026. The SBR framework is designed to be durable across near-term tool changes — the principles are stable; the specific AI tools and test frameworks are interchangeable.

A note on sources: Several citations in this paper reference industry survey reports, practitioner publications, and vendor documentation rather than peer-reviewed academic papers. This reflects the state of the field — empirical research on AI-assisted software development practices is only beginning to appear in peer-reviewed venues, and the most current quantitative data on AI coding velocity, code quality, and team adoption patterns is available primarily through industry research. Where peer-reviewed equivalents exist they have been used; where they do not, the cited sources have been selected for methodological transparency, sample size, and relevance to the specific assertion they support.